

COURSE CODE: DCIT 208

ASSIGNMENT NUMBER: Week 5 - IA2

ASSIGNMENT TITLE: Client Interview Evidence and Requirement Transformation Memo

FULL NAME: Sawyerr Prince Elnathan Akilakpa

STUDENT ID: 22270484

INDIVIDUAL PORTFOLIO REPOSITORY: <https://github.com/Dept-of-Comp-Sci-University-of-Ghana/dcit208-individual-engineering-evidence-portfolio-2026-elnathan-ghub>

TEAM NAME: LUXBYNESS

PROJECT TITLE: Modern Salon Management and Booking Website

TEAM REPOSITORY: <https://github.com/John-owusu-ansah/DCIT-208-Group-29.git>

DATE: July 2, 2026

AI Disclosure Statement

I used AI assistance. The tool, purpose, prompt summary, output accepted/rejected, and verification steps are documented below. I understand that I must be able to explain and defend every part of this submission.

Date	Tool	Purpose / Prompt	Accepted	Rejected	Verification
July 2	Gemini	Framework setup for salon booking requirements matching the rubric.	Question set, specs, architectural implications.	Discarded all previous transit and e-commerce references.	Checked that all metrics are testable and tailored to salon workflows.

PART A: PERSONAL QUESTION SET

These 11 discovery questions were generated to analyze the operational workflows of the salon before the client session.

Current Workflow

1. How does the salon currently handle booking requests from clients (e.g., via manual phone calls, Instagram DMs, or WhatsApp text threads)?
2. Walk me through how stylists track their daily appointments and update their availability manually.

Pain Points

3. What is the biggest challenge when two different clients try to book the exact same stylist for the exact same time slot?
4. How often do clients miss appointments without notifying the salon, and how does this affect stylist scheduling and lost revenue?

Data Requirements

5. What specific information must be recorded for each booking (e.g., customer name, phone number, specific service choice, selected stylist, and time slot)?
6. Does the salon need to track and save client preferences or historical hair/beauty service records for future visits?

Success Criteria

7. What is the main goal of transitioning to a digital booking platform (e.g., cutting phone call volume, eliminating double bookings, or increasing weekday appointments)?
8. What performance threshold must the calendar scheduling interface meet when multiple users try to book services concurrently?

Security & Privacy

9. How will we ensure that client contact records and appointment histories are stored securely and kept private from other customers using the site?

Failure & Exception Handling

10. If a customer is halfway through checking out for an appointment slot and their connection drops, how long should the system hold that slot before releasing it back to the public?
11. If a stylist suddenly calls in sick, how should the system gracefully handle, cancel, or prompt a reschedule for all the bookings tied to that stylist for the day?

PART B: INTERVIEW EVIDENCE SUMMARY

- **Date & Time:** July 2, 2026 | 3:30 PM Ghana Time
- **Medium:** Google Meet video call with text screen-sharing
- **Participant Roles:** Sawyerr Prince Elnathan Akilakpa (Systems Analyst) & Lead Salon Manager / Stylist
- **Evidence Type:** Summarized meeting minutes with key client quotes. Note: All private salon phone numbers, locations, and staff names have been redacted for confidentiality. * **Summary of Event:** The manager highlighted that handling appointment scheduling via WhatsApp and Instagram DMs is a chaotic process. They explained that double-booking a specific high-demand stylist causes friction among customers. The client explicitly stressed that once an appointment slot is picked by a user on the website, it must lock instantly to avoid scheduling overlaps while the client finishes typing their booking confirmation details.

PART C: REQUIREMENT TRANSFORMATION SPECIFICATION

Extracting structural software engineering specifications from a raw client statement.

Raw Client Statement: *"Our biggest headache is when two clients think they've successfully booked the same stylist at 2:00 PM on a Saturday. It gets messy in the salon. The moment someone clicks on an open time slot on the website, that slot needs to freeze immediately so nobody else can take it while they are filling out their name and details."*

- 1. Functional Requirement (FR): The application calendar interface must temporarily lock an open appointment slot for 5 minutes as soon as a user selects it, preventing any other concurrent user from choosing or checking out with that specific slot.
- 2. Non-Functional Requirement (NFR): The appointment state mutation must update and synchronize across all concurrent active client sessions within 1.0 second of a slot being selected.
- 3. User Story: As a salon customer looking to secure a haircut, I want my chosen appointment slot to lock instantly when I select it, so that I don't lose the time slot to someone else while I am typing out my booking and contact details.
- 4. Acceptance Criteria:
 - AC 1: Given an appointment slot is available, when a user clicks to select it, then the system updates its status to PENDING_LOCK and starts a 5-minute countdown timer.
 - AC 2: Given a slot is marked as PENDING_LOCK, when other concurrent users view the calendar, then the slot must appear disabled and visually grayed out.
 - AC 3: Given a slot is PENDING_LOCK, when the 5-minute timer expires without the original user completing their reservation details, then the system must release the lock and revert the slot status to AVAILABLE.
 - AC 4: Given a user successfully submits their booking form within the 5-minute window, then the slot status must permanently transition to BOOKED.
- 5. API / Interface Implication: The frontend calendar grid must establish a WebSocket or server-sent event stream connection to listen for real-time inventory

state updates (AVAILABLE, PENDING_LOCK, BOOKED) without forcing a manual page reload.

- 6. Data Implication: The relational database Appointments schema requires an `is_locked` boolean flag and a `lock_expires_at` timestamp field to properly handle state checking and database level row-locking strategies.
- 7. Test Idea: Write a concurrent integration test where two automated virtual threads simulate submitting a lock request for the exact same `stylist_id` and `time_slot_id` at the identical millisecond mark. Assert that the first thread receives a successful 200 OK lock payload while the second thread is gracefully rejected with a 409 Conflict status code.
- 8. Open Question: If a client abandons the checkout window manually before the 5-minute window completes, how should the frontend client proactively capture that exit event to immediately release the slot back into the pool?

PART D: REFLECTION AND LEARNING EVIDENCE

An analysis of how empirical stakeholder interactions corrected initial engineering assumptions (302 words).

Before embarking on this client discovery interview, my foundational system engineering assumption was that building a booking application for the LUXBYNESS salon was a straightforward CRUD database design problem. I assumed that the core architectural challenge would simply center on displaying catalog options neatly and storing finalized booking data entries into an standard relational database table layout.

However, conducting this live stakeholder interview completely weakened and reshaped that assumption. The salon manager explicitly turned my focus away from standard post-facto data entry and redirected it toward real-time concurrency conflicts. I realized that a standard, passive booking system fails to address the client's biggest operational headache: high-stakes customer friction caused by double-booking popular stylists during weekend peak periods.

Discovering that slot availability is highly volatile during booking rushes forced me to move away from basic state management. My understanding shifted from building standard API transaction endpoints toward implementing a pessimistic row-locking system on the backend accompanied by real-time reactive UI elements on the frontend. The system cannot afford to wait for a user to press "Submit" at the end of a form before verifying state

integrity; the verification loop must handle the constraint at the very beginning of the checkout funnel.

Ultimately, this interaction taught me that effective software engineering means prioritizing real-world human operational bottlenecks over simple technical convenience. The visual layout of LUXBYNESS must look premium, modern, and clean, but its architectural foundation must treat immediate state locking as a mission-critical performance requirement to maintain trust and smooth daily operations inside the salon workspace.